```
Gru code used: gru_model = Sequential([
    Input(shape=(X_train.shape[1], X_train.shape[2])),
    GRU(50, return_sequences=True),
    Dropout(0.2),
    GRU(50),
    Dropout(0.2),
    Dense(1)
])
gru_model.compile(optimizer=Adam(learning_rate=0.001), loss='mse', metrics=['mae'])
```

Models Performance Comparison:

	MAE	MSE	RMSE	R2	MAPE
LSTM	2.207361	12.606536	3.550568	0.819425	2.528746
GRU	**1.739216**	**7.691356**	**2.773329**	**0.889830**	**1.997658**
Deep Regression	2.225919	13.292469	3.645884	0.809600	2.563857
Sequential	1.918786	9.934344	3.151879	0.857701	2.201629

***Text box1**: GRU performed with few layers*

The **Deep Regression** model, designed as a CNN-LSTM hybrid with layers like Conv1D, MaxPooling1D, Bidirectional LSTMs, excelled effectively balancing regression and time-series capabilities. The **Sequential Dense** model, treating time instances independently, was modified with minor changes and its performance remained limited due to its inability to capture temporal dependencies.

2.1. **Models' Architectures:**

The following architectures were implemented:

2.1.1. LSTM Model

- Stacked **LSTM layers** to capture complex sequential dependencies in the input time series.
- **Dropout layers** after each LSTM for regularisation and to prevent overfitting.
- A **Dense output layer** to produce the final stock price prediction.
- Compiled with **Adam optimizer** and **MSE loss** for regression accuracy.

2.1.2. GRU Model

- Stacked **GRU layers** with L2 regularization to learn temporal dependencies and prevent overfitting.
- **Batch Normalization** layers to stabilize and accelerate training.
- **Dropout layers** for additional regularization and improved generalization.
- **Dense hidden layer** with ReLU activation for nonlinear transformation.
- Final **Dense output layer** for stock price prediction.
- Compiled with **Adam optimizer** and **MSE loss** for regression performance.

2.1.3. Deep Regression Model

- **Input layer** with the shape corresponding to the features in the dataset.
- **1D Convolutional** layer with 128 filters, kernel size of 3, ReLU activation, and L2 regularization.
- **MaxPooling** layer with pool size of 2.
- **Dropout layer** with a rate of 0.4.
- **Bidirectional** LSTM layer with 128 units and return sequences.
- **Dropout layer** with a rate of 0.4.
- **Bidirectional LSTM layer** with 64 units.

- **Dense layer** with 32 units, ReLU activation, and L2 regularization.
- **Output layer** with a single neuron predicting the stock price.

2.1.4. Sequential Dense Neural Network Model

- **Input layer** with the shape corresponding to the features in the dataset.
- **Flatten layer** to reshape the input.
- **Dense layer** with 64 units and ReLU activation.
- **Dropout layer** with a rate of 0.3.
- **Dense layer** with 32 units and ReLU activation.
- **Output layer** with a single neuron predicting the stock price.

3. Analysis of the Training Process:

The data was pre processed for the better evaluation of all models.

3.1.Data preparation:

The data preparation process involved:

3.1.1. Data Cleaning: Removed unnecessary columns (`Unnamed: 0`, `Time`).

3.1.2. Feature Scaling: Scaled stock price values using MinMaxScaler (ranged between 0 and 1) for optimal performance.

3.1.3. Creating Sequences: Transforming data into sequences (lookback window = 30 days) for time series forecasting.

3.1.4. Data Splitting:

Dataset	Percentage	Samples
Train	70 %	1260
Validation	15%	270
Test	15%	270

Table 1: Data splitting

After pre-processing the data, training was performed using **Mean Squared Error (MSE)** as the loss function and **Adam Optimiser** for weight updates.

3.1.5. Key aspects of the training process are:

- **Early Stopping:** Implemented to prevent overfitting by monitoring validation loss.
- **Batch size:** 32
- **Epochs:** 100

3.2. Best Epoch Metrics

Metrics	LSTM	GRU	Deep Regression	Sequential Dense
Best Epoch	41	60	39	6
Val Loss	0.0011	0.0068	**0.0013**	0.0126
Val MAE	0.0252	0.0289	**0.0230**	0.1081
Convergence	Fast	Slower	Moderate	Moderate
Stability	High	Gradual, Improving	High, Steady	Moderate, Improving

Table 2: comparison of the best epoch matrix for the models

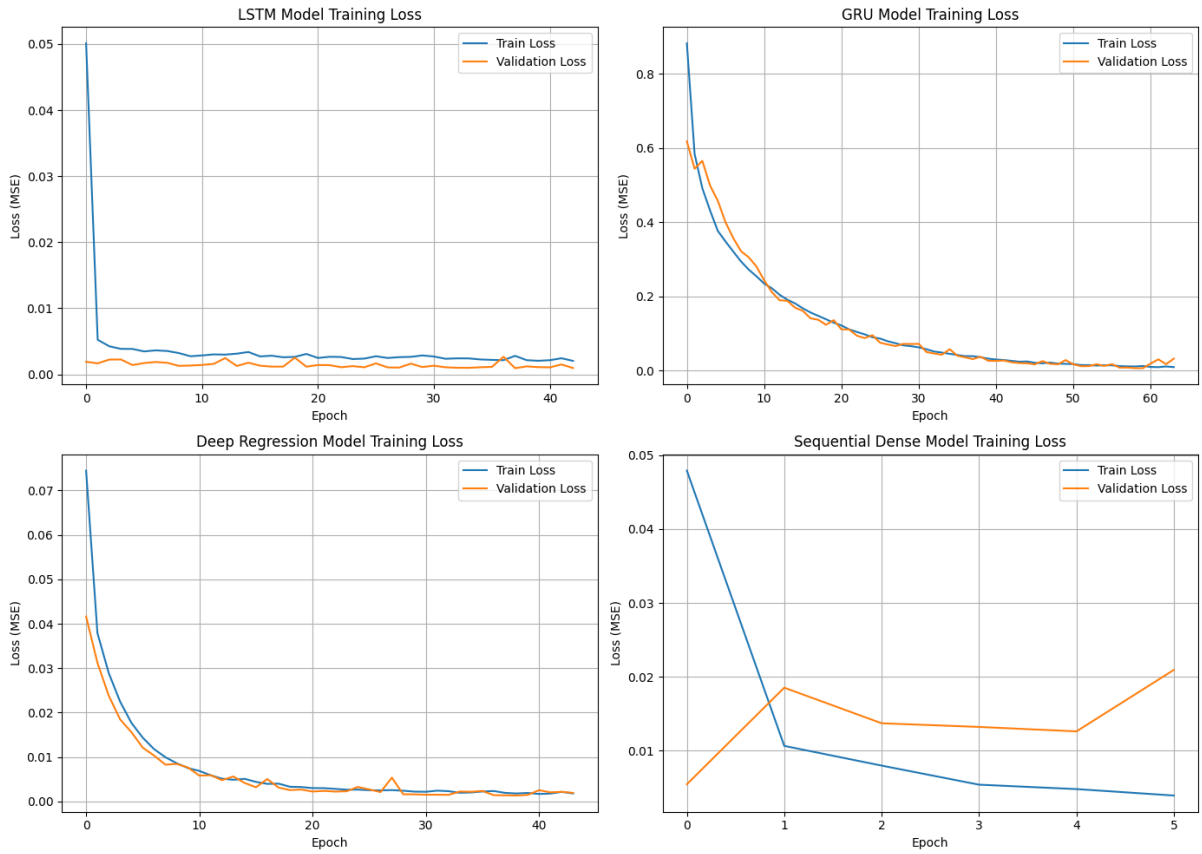
Due to **early stopping, none** of the models reached **100 epochs.** Based on the best-epoch validation metrics, **Deep Regression and LSTM models** demonstrated early convergence and lower validation errors compared to GRU and Sequential models as shown in table 2. **Deep Regression** showed strong

performance in terms of **validation MAE** and stability, the **LSTM** model achieved the **lowest validation Loss** very closed to **Deep Regression**, indicating the most accurate predictions at its optimal epoch. **Only 6 epochs** are extremely short training for a complex model like Sequential Dense.

4. Comparative Analysis and Performance Evaluation

4.1.Training Loss Curves:

The following loss curves depict the training and validation Mean Squared Error (MSE) over epochs for all four deep learning models:



Graph 1: the loss curves of all models

The loss curves for the LSTM, GRU, Deep Regression, and Sequential Dense models reveal distinct training dynamics as shown in graph 1. The **LSTM** model converges rapidly within 5 epochs to a low validation loss (0.005) with **no overfitting**, indicating efficient learning and strong generalization. In contrast, the **GRU** model exhibits slow convergence (30 epochs) to a higher validation loss (0.1) with mild **overfitting** (training loss 0.05), suggesting challenges in generalization despite extended training to 60 epochs. The **Deep Regression** model converges quickly (5 epochs) to a validation loss of 0.01 but shows **slight instability** (spikes around epochs 15–20), indicating minor generalization issues. The **Sequential Dense** model converges fastest (2 epochs) to a validation loss of 0.01 with minimal overfitting, though its **higher loss** compared to LSTM reflects limited learning capacity. Overall, LSTM demonstrates the best training performance, while GRU struggles with overfitting, and Deep Regression and Sequential Dense balance speed and stability with moderate performance.

4.2.Models' Performance: Training and Test Metrics

The following tables summarise the performance of the models on the training and test sets, using Mean Absolute Error (MAE), Mean Squared Error (MSE), Root Mean Squared Error (RMSE), R^2 (coefficient of determination), and Mean Absolute Percentage Error (MAPE) as evaluation metrics. These metrics are complemented by the previously analysed loss curves to provide a holistic view of each model's behaviour.

Training Performance:					
Models	**MAE**	**MSE**	**RMSE**	**R^2**	**MAPE**
LSTM	1.689336	5.614998	2.369599	0.985054	2.061995
GRU	5.301139	39.664537	6.297979	0.894418	6.782436
Deep Regression	**1.681162**	**5.336220**	**2.310026**	**0.985796**	**2.037523**
Sequential Dense	4.415293	24.877340	4.987719	0.933779	5.206284
Test Performance:					
LSTM	1.878591	9.284584	3.047062	0.867008	2.158326
GRU	4.177462	30.967218	5.564820	0.556428	4.915681
Deep Regression	**1.721462**	**7.723795**	**2.779172**	**0.889365**	**1.973590**
Sequential Dense	4.723111	28.825076	5.368899	0.587112	5.236988

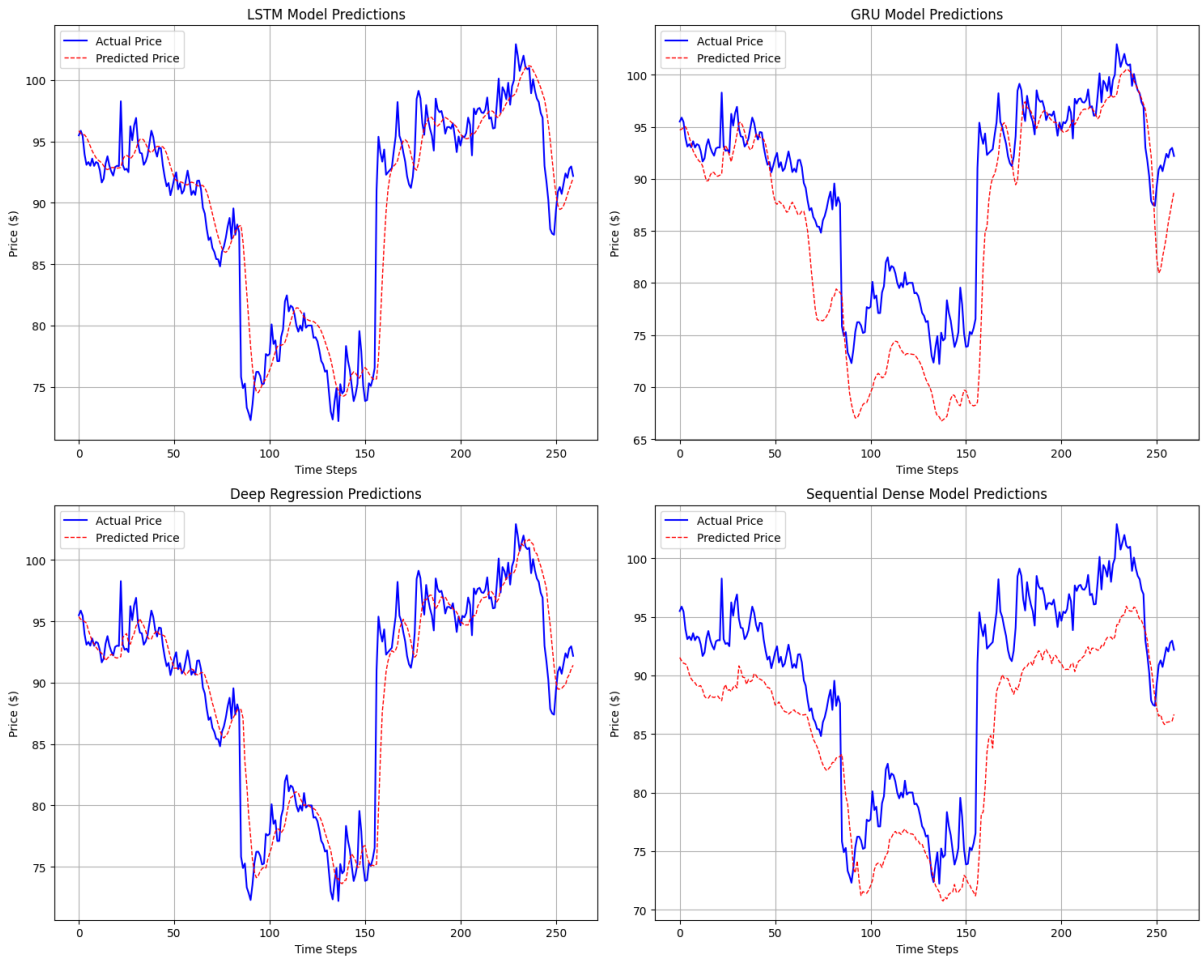
Table 3: Performance of the models on the basis of different matrices

As shown in the table, the **Deep Regression** model excels on both training (R^2: 0.985796) and test sets (R^2: 0.889365), with the lowest test errors (MAE: 1.721462, MAPE: 1.973590). Its slight instability in the loss curves does not significantly impact generalization, making it the **best model** for deployment. Despite regularisation (L2 and dropout), the **GRU** model **overfits** (validation loss 0.1 vs. training loss 0.05), leading to poor test performance (R^2: 0.556428). This contrasts with its earlier metrics (MAE: 1.831548, R^2: 0.893187), suggesting changes in data pre-processing, model architecture, or training process may have impacted its performance negatively.

The **LSTM** model performs well on both training (R^2: 0.985054) and test sets (R^2: 0.867008), with **minimal overfitting**. However, it is outperformed by Deep Regression on the test set, possibly due to over-optimization on the training or validation data. The **Sequential Dense** model's fast convergence is a strength, but its simpler architecture results in **higher errors** and poor generalization (test R^2: 0.587112), making it less suitable for this task.

4.3.Model' Predictions vs. Actual Stock Prices

The following plots compare the actual Starbucks stock prices (blue) with the predicted prices (red dashed) over 250 time steps for the models:



Graph 2: Model' Predictive vs. Actual Stock Prices

The **Deep Regression** model's predictions are the most accurate, **closely tracking actual prices** and handling volatility well as shown in the graph 2. This aligns with its best test performance (R^2: 0.889365, MAE: 1.721462), making it the top choice for forecasting and the deployment. The **LSTM** model also performed well, with predictions closely following actual prices, though **slightly less accurately** than Deep Regression. Its good test metrics (R^2: 0.867008) and low validation loss make it a reliable alternative.

The **GRU** model's predictions **deviate significantly** from actual prices, reflecting its poor test performance (R^2: 0.556428) and overfitting observed in the loss curves. This indicates it is not suitable for this task in its current form. The **Sequential Dense** model captures general trends but consistently **underestimates prices** and fails to handle volatility, aligning with its poor test performance (R^2: 0.587112). Its simpler architecture limits its effectiveness for time-series forecasting.

5. Conclusion and Recommendations:

The analysis of the models reveals distinct strengths and weaknesses for each model, leading to a clear recommendation for deployment and targeted improvements.

The **Deep Regression** model demonstrates superior performance, achieving the lowest test errors (MAE: 1.72, RMSE: 2.78) and highest predictive accuracy (R^2: 0.89), while its loss curves confirm stable convergence with minimal overfitting. Its predictions closely match actual price movements, even during volatile periods. The **LSTM** shows competent but slightly weaker results (R^2: 0.87), with good trend capture but minor lagging during market swings. Both **GRU and Sequential Dense** models underperform significantly. GRU struggles with high errors (MAE: 4.18, R^2: 0.56) and systematic underestimation, while the Dense model's simplicity leads to consistent prediction bias (MAE: 4.72). These results clearly establish Deep Regression as the optimal choice for the starbucks stock price forecast and deployment, with LSTM serving as a viable alternative, while the GRU and Dense architectures require substantial revision to be considered effective forecasting tools.

In conclusion, the **Deep Regression model** is recommended for deployment in real-time financial forecasting of Starbucks stock prices, given its superior predictive accuracy, robust generalization, and ability to capture both trends and volatility. The LSTM model serves as a viable alternative for scenarios requiring minimal overfitting and efficient training.

Future work should focus on adding more features (e.g., lagged prices, market indicators and news sentiments) and analysing prediction errors during volatile periods to further improve all models, ensuring more accurate and reliable financial forecasting.

6. References

i. StatQuest with Josh Starmer (2015) 'Getting Started with Keras'. Available at: https://www.youtube.com/watch?v=J6Ok8p463C4 (Accessed: 25 March 2025).

ii. deeplearning.tv (2015) 'Restricted Boltzmann Machines - Ep. 6 (Deep Learning SIMPLIFIED)'. Available at: https://www.youtube.com/watch?v=puux7KZQfsE (Accessed: 20 March 2025).YouTube

iii. NeuralNine (2020) 'Tensorflow Tutorial for Python in 10 Minutes'. Available at: https://www.youtube.com/watch?v=6_2hzRopPbQ (Accessed: 28 April 2025).YouTube

iv. TensorFlow (2024) 'Basic regression: Predict fuel efficiency'. Available at: https://www.tensorflow.org/tutorials/keras/regression (Accessed: 28 March 2025). TensorFlow

v. TensorFlow (2024) 'Time series forecasting'. Available at: https://www.tensorflow.org/tutorials/structured_data/time_series (Accessed: 1 April 2025). TensorFlow

vi. TensorFlow (2024) 'tf.keras.layers.LSTM'. Available at: https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM (Accessed: 1 April 2025). TensorFlow

vii. TensorFlow (2024) 'tf.keras.layers.GRU'. Available at: https://www.tensorflow.org/api_docs/python/tf/keras/layers/GRU (Accessed: 6 April 2025).

viii. Li, Y. and Pan, Y. (2022) 'A novel ensemble deep learning model for stock prediction based on stock prices and news', International Journal of Data Science and Analytics, 13, pp. 139–149. Available at: https://doi.org/10.1007/s41060-021-00279-9 (Accessed: 6 April 2025). SpringerLink

ix. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y. (2014) ‘Learning phrase representations using RNN encoder-decoder for statistical machine translation’, *arXiv preprint*, arXiv:1406.1078.

x. Hochreiter, S. and Schmidhuber, J. (1997) 'Long short-term memory', *Neural Computation*, 9(8), pp. 1735–1780.

xi. Fischer, T. and Krauss, C. (2018) 'Deep learning with long short-term memory networks for financial market predictions', *European Journal of Operational Research*, 270(2), pp. 654–669.

xii. Chung, J., Gulcehre, C., Cho, K. and Bengio, Y. (2014) 'Empirical evaluation of gated recurrent neural networks on sequence modeling', *arXiv preprint*, arXiv:1412.3555.

xiii. Borovykh, A., Bohte, S. and Oosterlee, C. W. (2017) 'Conditional time series forecasting with convolutional neural networks', *arXiv preprint*, arXiv:1703.04691.
Shi, X., Chen, Z., Wang, H., Yeung, D. Y., Wong, W. and Woo, W. (2015) 'Convolutional LSTM network: A machine learning approach for precipitation nowcasting', *arXiv preprint*, arXiv:1506.04214.

xiv. Galaxy Training Network (2021) 'Deep Learning (Part 1) - Feedforward neural networks (FNN)'. Available at: https://training.galaxyproject.org/training-material/topics/statistics/tutorials/FNN/tutorial.html (Accessed: 6 April 2025).

xv. Goodfellow, I., Bengio, Y. and Courville, A. (2016) *Deep Learning*. Cambridge, MA: MIT Press.

xvi. Chollet, F. (2021) *Deep Learning with Python*. 2nd edn. Shelter Island, NY: Manning Publications.

7. APPENDIX II – Use of Generative AI

This submission represents our own independent work, and all external sources have been appropriately referenced. Generative AI was USED during the preparation of this submission, as indicated in the references provided. A full log of prompts submitted to the AI tool and the corresponding responses is included below.

with my these codes i am getting future warning what i can do: from sklearn.preprocessing import MinMaxScaler scaler = MinMaxScaler(feature_range=(0, 1)) scaler.fit(train_data[['Open']]) train_data.loc[:, 'Open'] = scaler.transform(train_data[['Open']]) val_data.loc[:, 'Open'] = scaler.transform(val_data[['Open']]) test_data.loc[:, 'Open'] = scaler.transform(test_data[['Open']]) print(f"Train size: {len(train_data)}, Validation size: {len(val_data)}, Test size: {len(test_data)}")

ChatGPT said:

Your **FutureWarning** is caused by the way you're modifying a Pandas DataFrame column using **chained assignment**. The issue occurs because `scaler.transform()` returns a **NumPy array**, and Pandas is warning that in future versions, this kind of assignment may not behave as expected.

✅Fix: Assign Values Correctly

Use `.iloc` or **convert to a Pandas Series** to avoid the warning.

✅ *Option 1: Use `.iloc` for Safe Assignment*

```
python
CopyEdit
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 1))
scaler.fit(train_data[['Open']])

# Assign values safely
train_data.iloc[:, train_data.columns.get_loc('Open')] =
scaler.transform(train_data[['Open']]).flatten()
val_data.iloc[:, val_data.columns.get_loc('Open')] =
scaler.transform(val_data[['Open']]).flatten()
test_data.iloc[:, test_data.columns.get_loc('Open')] =
scaler.transform(test_data[['Open']]).flatten()

print(f"Train size: {len(train_data)}, Validation size: {len(val_data)},
Test size: {len(test_data)}")
```

✅ `.iloc` is a safe way to update values without triggering warnings.

✅ *Option 2: Convert to Pandas Series*

```
python
CopyEdit
```

```
from sklearn.preprocessing import MinMaxScaler
scaler = MinMaxScaler(feature_range=(0, 1))
scaler.fit(train_data[['Open']])

# Convert transformed values to a Pandas Series with index
train_data['Open'] =
pd.Series(scaler.transform(train_data[['Open']]).flatten(),
index=train_data.index)
val_data['Open'] =
pd.Series(scaler.transform(val_data[['Open']]).flatten(),
index=val_data.index)
test_data['Open'] =
pd.Series(scaler.transform(test_data[['Open']]).flatten(),
index=test_data.index)

print(f"Train size: {len(train_data)}, Validation size: {len(val_data)},
Test size: {len(test_data)}")
```

 This approach ensures the transformed NumPy array aligns with the DataFrame's index.

Why Do These Fixes Work?

- `scaler.transform()` returns a **NumPy array** (2D by default).
- Pandas expects **proper alignment** when modifying DataFrame columns.
- `.flatten()` ensures it is a **1D array**, preventing mismatched shapes.
- Using `.iloc` or converting to `pd.Series` ensures explicit assignment **without ambiguity**.

Your **FutureWarning** should now disappear! Let me know if you need further clarification.

4o

/usr/local/lib/python3.11/dist-packages/keras/src/layers/rnn/rnn.py:200: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
super().__init__(**kwargs)

ChatGPT said:

The warning you're seeing is related to how you've defined the input shape for your RNN layers (like `LSTM` or `GRU`) within your Keras model.

What the warning means:

- The warning is telling you that when you define a Sequential model in Keras, it is **recommended** to use an `Input(shape)` layer as the first layer of the model, instead of directly passing the `input_shape` to the RNN layer (e.g., `LSTM` or `GRU`).

How to fix the warning:

1. **Use `Input(shape)` as the first layer** instead of passing `input_shape` directly to the RNN layer.

   This makes the model definition more clear and avoids the warning. It's a more modern and flexible way of defining the input shape.

Update your code:

In your model definition, you're likely defining something like this:

```
python
CopyEdit
lstm_model = Sequential([
    LSTM(50, return_sequences=True, input_shape=(X_train.shape[1],
X_train.shape[2])),
    Dropout(0.2),
    LSTM(50),
    Dropout(0.2),
    Dense(1)
])
```

To avoid the warning, you should change it to use an `Input` layer:

```
python
CopyEdit
from tensorflow.keras.layers import Input

lstm_model = Sequential([
    Input(shape=(X_train.shape[1], X_train.shape[2])),  # Use Input layer
here
    LSTM(50, return_sequences=True),
    Dropout(0.2),
    LSTM(50),
    Dropout(0.2),
    Dense(1)
])
```

Explanation:

- The `Input(shape=...)` layer serves as the first layer of the model and defines the shape of the input data.
- After that, you can stack your RNN layers (`LSTM`, `GRU`, etc.) without specifying the input shape again.
- This approach is more flexible and is the recommended way to define the input layer in Keras, especially when using the `Sequential` model.

Conclusion:

To fix the warning, just replace the `input_shape` argument in the RNN layer with an `Input(shape=...)` layer at the beginning of your model. This will prevent the warning from being displayed and keep your code up-to-date with the latest Keras practices.

4o mini

open these links and give me the harvard style references:
https://youtu.be/J6Ok8p463C4?si=H5sxzWavqbVspxV4
https://youtu.be/puux7KZQfsE?si=pQCHDx6cFskN0Wb1
https://youtu.be/6_2hzRopPbQ?si=QjQAOBDlPHTcNrRc
https://www.tensorflow.org/tutorials/keras/regression
https://www.tensorflow.org/tutorials/structured_data/time_series
https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM
https://www.tensorflow.org/api_docs/python/tf/keras/layers/GRU
https://training.galaxyproject.org/training-material/topics/statistics/tutorials/FNN/tutorial.html
https://link.springer.com/article/10.1007/s41060-021-00279-9

ChatGPT said:

Certainly! Here are the Harvard-style references for the provided links:

1. StatQuest with Josh Starmer (2015) 'Getting Started with Keras'. Available at: https://www.youtube.com/watch?v=J6Ok8p463C4 (Accessed: 9 April 2025).
2. deeplearning.tv (2015) 'Restricted Boltzmann Machines - Ep. 6 (Deep Learning SIMPLIFIED)'. Available at: https://www.youtube.com/watch?v=puux7KZQfsE (Accessed: 9 April 2025).YouTube
3. NeuralNine (2020) 'Tensorflow Tutorial for Python in 10 Minutes'. Available at: https://www.youtube.com/watch?v=6_2hzRopPbQ (Accessed: 9 April 2025). YouTube
4. TensorFlow (2024) 'Basic regression: Predict fuel efficiency'. Available at: https://www.tensorflow.org/tutorials/keras/regression (Accessed: 9 April 2025). TensorFlow
5. TensorFlow (2024) 'Time series forecasting'. Available at: https://www.tensorflow.org/tutorials/structured_data/time_series (Accessed: 9 April 2025).TensorFlow
6. TensorFlow (2024) 'tf.keras.layers.LSTM'. Available at: https://www.tensorflow.org/api_docs/python/tf/keras/layers/LSTM (Accessed: 9 April 2025).TensorFlow
7. TensorFlow (2024) 'tf.keras.layers.GRU'. Available at: https://www.tensorflow.org/api_docs/python/tf/keras/layers/GRU (Accessed: 9 April 2025).
8. Galaxy Training Network (2021) 'Deep Learning (Part 1) - Feedforward neural networks (FNN)'. Available at: https://training.galaxyproject.org/training-material/topics/statistics/tutorials/FNN/tutorial.html (Accessed: 9 April 2025).
9. Li, Y. and Pan, Y. (2022) 'A novel ensemble deep learning model for stock prediction based on stock prices and news', International Journal of Data Science and Analytics, 13, pp. 139–149. Available at: https://doi.org/10.1007/s41060-021-00279-9 (Accessed: 9 April 2025).SpringerLink

early stopping code for stock forecasting models

ChatGPT said:

Early stopping is a great regularization technique to prevent overfitting when training models like LSTMs, GRUs, or other neural nets for stock price forecasting. Below is a simple implementation of early stopping in **PyTorch** and **TensorFlow/Keras**, so you can use whichever you're working with.

□ 1. PyTorch Early Stopping Class (for Stock Forecasting)

```
python
CopyEdit
class EarlyStopping:
    def __init__(self, patience=5, min_delta=0.001, verbose=True):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_loss = None
        self.early_stop = False
        self.verbose = verbose

    def __call__(self, val_loss):
        if self.best_loss is None:
            self.best_loss = val_loss
        elif val_loss > self.best_loss - self.min_delta:
            self.counter += 1
            if self.verbose:
                print(f"EarlyStopping counter: {self.counter} out of
{self.patience}")
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_loss = val_loss
            self.counter = 0
```

Usage Example:

```
python
CopyEdit
early_stopping = EarlyStopping(patience=10)

for epoch in range(num_epochs):
    train(...)
    val_loss = validate(...)

    early_stopping(val_loss)

    if early_stopping.early_stop:
        print("Stopping early!")
        break
```

□ 2. TensorFlow/Keras Early Stopping

You don't even need to implement it yourself — Keras has it built-in:

```
python
CopyEdit
from tensorflow.keras.callbacks import EarlyStopping

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=10,
    min_delta=0.001,
    restore_best_weights=True,
    verbose=1
)

model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=100,
    callbacks=[early_stop],
    batch_size=32
)
```

If you're building a stock prediction model with LSTM or GRU, both options above work perfectly — just tailor the `monitor` metric and `patience` based on your validation loss curve.

Need a full example with a stock price forecasting model too?

4o

You said:
if i use patient 5 instead of 10 how it will impact my model

ChatGPT said:

Great question — changing the `patience` parameter from **10 to 5** will directly impact how sensitive early stopping is to a lack of improvement in your model's validation performance.

□ Here's what changes with `patience=5` vs `10`:

Setting	Description	Impact on Model
`patience=10`	Waits longer (10 epochs) for val loss to improve	Gives model more chances to improve; might overfit slightly more
`patience=5`	Waits less (5 epochs) for val loss to improve	Stops training sooner; might prevent overfitting earlier

---

✅Benefits of Lower Patience (e.g., 5)

- **Faster training** – saves time if your model plateaus early.
- **Prevents overfitting** – less chance to keep training after hitting the validation loss floor.

- **Useful for volatile datasets** – like stocks, where small noise can look like improvement.

□ Possible Downsides

- **Might stop too early** if your model has a slow convergence pattern or you're using a learning rate schedule.
- You **could miss out on slight improvements** that appear after a few more epochs.

□ When should you use `patience=5`?

- When you're seeing **very early convergence** in validation loss.
- If you're experimenting with many models and need to **speed up training cycles**.
- When **overfitting happens quickly** — especially with smaller datasets.

Would you like me to help you plot validation loss with different patience values to visualize the impact?

4o